WISEConv: A Worklist-driven Masked-Convolution Runtime for GPUs

Anonymous Authors

***Abstract*—Masked convolution promises to cut the per-frame inference latency of vision models by evaluating only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to convert sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime that derives an active-output worklist from the upstream mask and uses it to drive the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by computing only needed positions while executing active positions through the regular dense datapath. To amortize construction overhead and keep the compute pipeline fully utilized despite per-frame-varying worklist lengths, WISEConv co-designs construction and consumption: compatible operators reuse the constructed worklist without reconstruction, the worklist emission order preserves tile-local spatial reuse, and a data-driven adaptive decomposition adjusts parallel work to the activity level without host synchronization. Across four perception models and six GPU platforms (four discrete, two embedded), WISEConv reduces full-model latency by 22–46% relative to the fastest competing path and per-frame energy by 28–60% on the embedded platforms.**

***Index Terms*—Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference.**

I. Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [1]–[3], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks the active positions, and the operator evaluates only those. The mask comes from sources including event-camera increments [4], [5], temporal differences between video frames [6], [7], and learned spatial gates [8], [9]. Regardless of the source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates consequently share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution selects positions at runtime per frame, requiring the operator to access scattered, irregularly-overlapping neighborhoods and handle a workload that varies per frame, seemingly conflicting with tile-level execution.

Existing masked-convolution systems preserve this regularity or eliminate wasted work, but not both. *Tile skipping* [4], [6], [7], [10] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [8], [9], [11] instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes (§II-B): *throughput* (operations per second) and *useful-compute ratio* (the fraction of executed operations that produce needed outputs). In our sparsest workload [8] (§V-B), the masks render 84.4% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [6] suffers a useful-compute ratio of only 0.49 and cuts latency by at most 41%. Gather-scatter drives the useful-compute ratio near one but sustains only 9% of tile skipping's throughput, running 3.1× slower than dense convolution.

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are evaluated without changing the per-position reduction, the dense tensor layout, or the weight layout. With only the coordinate source changed, the operator runs as a dense tiled pipeline, pinning the useful-compute ratio near one without input-patch materialization.

Translating this into latency reduction requires handling two challenges. First, *worklist-construction overhead*: building the worklist scans the mask, so its cost scales with the layer's spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction's share of latency therefore grows as activity falls and channels thin, and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist's order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-dr**I**ven ma**SkE**d **Conv**olution), a runtime for masked convolution that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is made cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators propagate and reuse valid metadata instead of reconstructing it. Consumption is made dense-like: construction

Manuscript received Month DD, YYYY.

emits contiguous per-tile segments ordered by the dense tile traversal, giving compute tile-local spatial structure without a global sort. Compute then decomposes parallel work adaptively over the spatial and reduction dimensions according to device, layer shape, and activity, while each selected position keeps the regular reduction over channels and kernel offsets.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into tiled dense convolution and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across four discrete GPUs and two Jetson platforms. WISEConv reduces full-model latency relative to the fastest baseline by 34–46% on the discrete GPUs and 22–39% on the Jetson platforms, and cuts per-frame energy over dense convolution by 28–60% on the Jetson platforms.

II. Background and Motivation

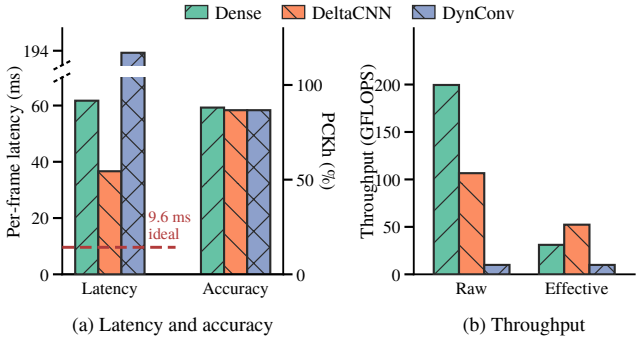
A. Masked Convolution over Dynamic Spatial Sparsity

Convolution runs inside the perception-action loop of many vision applications [1], [2], [12]–[14], where per-frame inference latency bounds system responsiveness [3], [15]–[17]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator updates only the marked positions. The mask comes from sources including event-camera increments [4], [5], temporal residuals between video frames [6], [7], [18]–[20], learned input-dependent gates [8], [9], [11], [21]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the dense tensors and arithmetic of a dense convolution and restricts only which output coordinates it evaluates. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in}\times W_{in}\times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k\times k\times C_{in}\times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out}\times W_{out}\times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out}W_{out}\}$, where each coordinate p has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out}\times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in}\times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus fixes which positions of $\mathbf{y}$ the operator updates, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$. The active ratio $\alpha = \|M_{out}\|_0/|\mathcal{G}|$, the active fraction of the grid, is the intrinsic sparsity the policy exposes.



(a) Latency and accuracy (b) Throughput

Fig. 1. Statistics of DynConv [8], a pose estimator with learned spatial gating, on the MPII human-pose dataset [22] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; PCKh evaluates the percentage of keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff}, aggregated across the model's convolution layers. The dense useful-compute ratio implies the active ratio set by DynConv's spatial gating ($\alpha \approx 15.6\%$).

An execution path realizes this mask over a compute set $\mathcal{C}$ bounded by $\{p : M_{out}[p] = 1\} \subseteq \mathcal{C} \subseteq \mathcal{G}$; for each $p \in \mathcal{C}$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i\, \mathbf{x}[\mathcal{N}_i(p)], \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in}\times C_{out}}$ the corresponding weight slice. The path must compute every active coordinate to avoid dropping a requested output but may also compute inactive ones; dense convolution is the extreme $\mathcal{C} = \mathcal{G}$. The policy alone fixes the accuracy and the required work, while the choice of $\mathcal{C}$ is determined by the path.

B. The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [4], [7], [10], [19], [20], [23] inherits the dense pipeline's tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [6] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [9], [11] instead extracts exactly the active coordinates, computes, and scatters them back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [8] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

We measure each path by two quantities. Its useful-compute ratio is the fraction of computed coordinates that are active,

$$\eta = \frac{\|M_{out}\|_0}{|\mathcal{C}|} \le 1. \tag{3}$$

Its throughput is the rate at which it processes issued FLOPs,

$$T = \frac{2\,|\mathcal{C}|\,k^2 C_{in} C_{out}}{t}, \tag{4}$$

where t is the per-frame latency and the leading factor of 2 counts each multiply-accumulate as two FLOPs. For the required $2\,\|M_{out}\|_0\,k^2 C_{in} C_{out}$ FLOPs, latency is set by the **effective throughput** $T_{\text{eff}} = \eta T$, the rate at which a path computes needed outputs.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [6]) reaches 36.6 ms, still 3.8× the floor, and gather-scatter, even with DynConv built to amortize its overhead, runs at 194 ms, far slower than dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 0.49; gather-scatter lifts the useful-compute ratio to one, yet extraction and data movement collapse its throughput to 5% of dense (Fig. 1b). Neither path efficiently converts the GPU's capability into effective throughput.

C. Distinction from Related Sparse Execution

Related sparse-execution work differs from WISEConv's masked convolution in representation, operator, sparsity type, or platform. 3D sparse convolution (SpConv v2 [24], TorchSparse++ [25], [26], Minuet [27], MinkowskiEngine [28], submanifold sparse convolution [29]) is built to organize computation over an unordered list of active voxels, constructing coordinate maps that route irregular input coordinates to outputs; its input is sparse by construction rather than a dense grid under a runtime mask, so it is related infrastructure rather than a baseline. Sparse matrix multiplication (DTC-SpMM [30], MaxK-GNN [31], Sputnik [32]) reorganizes a sparse matrix operand so the dense datapath can consume it; its sparsity lives in the matrix structure of a different operator, not in a spatial mask over a convolution grid. Structured weight sparsity (2:4 tensor cores [33]) and sparse-format compilers (TACO [34], SparseTIR [35]) exploit a static structural sparsity fixed at compile time in the weights or a known format, rather than a mask discovered at runtime over the spatial grid. DPACS [36] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

III. WISEConv Design

A. Overview

WISEConv replaces the coordinate source of a dense tiled convolution with an active-output worklist derived from the upstream mask. The operator runs in two stages: a *construction stage* that turns the mask into a worklist of active output coordinates, and a *compute stage* that drives a tiled convolution from that worklist. Because our target is per-frame inference latency (whether a frame runs alone or is batched across independent sequences), every design decision below is driven by minimizing per-frame overhead.

To translate position-level selectivity into latency reduction, construction must be cheap to perform and the worklist must be efficient to consume. The design addresses both through the following choices.

a) Construction.: Construction scans the mask to discover active coordinates before compute can begin. To recover spatial locality for compute without a separate sorting pass, construction traverses the output grid in fixed tiles and compacts each tile's active coordinates in place. Each emitted segment inherits the tile's dense traversal order, so coordinates within a segment address nearby positions whose receptive fields overlap. Compute can therefore exploit tile-local data reuse without requiring a globally sorted worklist (§**??**).

Because construction cost is fixed for a given grid size, repeating it at every operator accumulates overhead that grows relative to the compute it enables. Consecutive operators that preserve the spatial grid (1×1 convolutions, activations, normalization) produce an identical worklist, so reconstruction is redundant. WISEConv lets these operators inherit the upstream worklist directly; construction executes only at geometry-change boundaries, amortizing its cost across compatible segments (§**??**).

b) Consumption.: The worklist is shorter than the dense grid and its length varies across frames and layers. The compute path must adapt to this variation without host synchronization, since a device-to-host sync per frame would stall the streaming inference pipeline. WISEConv launches a fixed, occupancy-sized persistent grid; each kernel reads the active count from device memory at entry and iterates over the worklist with a grid-stride loop, handling any length without a host-side launch decision (§**??**).

Beyond launch sizing, the parallel decomposition that best utilizes the GPU changes with the activity level: the tradeoff between per-tile compute efficiency and device occupancy shifts as the number of active positions varies (e.g., low activity may require additional parallelism to keep the device occupied). Searching for such a decomposition online is too expensive, and reading per-layer activity to select it would reintroduce the synchronization. WISEConv adopts a data-driven approach, exploiting the temporal correlation of streaming inputs: it reads back the input activity once per frame via an asynchronous device-to-host copy, lagged by one frame but never blocking. That single readback selects a pre-calibrated activity bucket; a per-layer mapping derived from data-driven offline calibration then supplies every operator's configuration without additional kernel launches or synchronization (§**??**).

IV. Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates

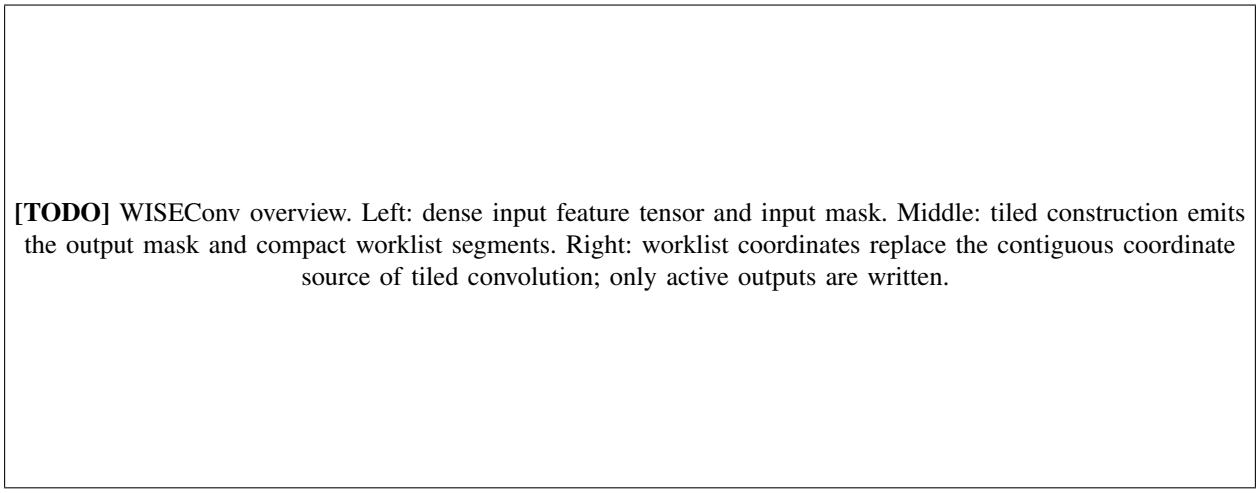


Fig. 2. The WISEConv two-stage pipeline. Construction discovers active output coordinates and compacts each spatial tile into a worklist segment. Compute uses the resulting coordinates to drive a tiled convolution while retaining its regular channel and kernel reduction.

once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section **??** without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split-K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split-K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight preparation complete before the timed sequence described in Section V-A.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

V. Evaluation

A. Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

1) Platforms: We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

2) Models, Tasks, and Mask Sources: Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [37], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [38]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [4], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on `indoor_flying1` and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [39] on MOT16 [40], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [6] supplies the

temporal mask mechanism by propagating thresholded activation deltas between successive frames.

- **Human pose estimation.** We use the DynConv pose model [8] on MPII [22], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table I summarizes their models and evaluation scales. We use official weights for all four models [8], [37], [39]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

TABLE I
EVALUATION WORKLOADS.

	FireFlowNet [37]	**YOLOv8n/m** [39]	**DynConv Pose** [8]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [4]	DeltaCNN [6]	DynConv [8]
Dataset	MVSEC [38]	MOT16 [40]	MPII [22]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

3) Execution Baselines: We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [41] with sparse execution disabled.
- **Tile skipping** follows the tile-sparse execution established by SBNet, Ev-Conv, and DeltaCNN [4], [6], [10].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [8] and an implementation equivalent to MMCV `MaskedConv2d` for FireFlowNet and YOLO [42].

4) Metrics and Methodology: Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on the model's compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work}, useful-compute ratio η, convolution throughput, and the construction fraction of operator latency. Because layers differ in cost, an unweighted average of layer activities misstates the work a mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\mathrm{work}} = \frac{\sum_l \rho^l W^l_{\mathrm{dense}}}{\sum_l W^l_{\mathrm{dense}}}, \qquad \rho^l = \frac{\|M^l\|_0}{H^l_{out} W^l_{out}}, \tag{5}$$

where W^l_{dense} is the dense MAC count of layer l. Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every requested output.

B. Full-Model Latency

C. Efficiency Analysis

D. Construction Overhead

E. Task Accuracy

ACKNOWLEDGMENTS

REFERENCES

[1] R. J. Bouwmeester, F. Paredes-Vallés, and G. C. H. E. de Croon, "Nanoflownet: Real-time dense optical flow on a nano quadcopter," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 1996–2003.

[2] D. Wofk, F. Ma, T. Yang, S. Karaman, and V. Sze, "Fastdepth: Fast monocular depth estimation on embedded systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6101–6108.

[3] D. Falanga, S. Kim, and D. Scaramuzza, "How fast is too fast? the role of perception latency in high-speed sense and avoid," *IEEE Robotics Autom. Lett.*, vol. 4, no. 2, pp. 1884–1891, 2019. [Online]. Available: https://doi.org/10.1109/LRA.2019.2898117

[4] S. Durvasula, Y. Guan, and N. Vijaykumar, "Ev-conv: Fast CNN inference on event camera inputs for high-speed robot perception," *IEEE Robotics Autom. Lett.*, vol. 8, no. 6, pp. 3174–3181, 2023.

[5] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, "Event-based asynchronous sparse convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 415–431.

[6] M. Parger, C. Tang, C. D. Twigg, C. Keskin, R. Wang, and M. Steinberger, "Deltacnn: End-to-end CNN inference of sparse frame differences in videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 12 487–12 496.

[7] A. Habibian, D. Abati, T. S. Cohen, and B. E. Bejnordi, "Skip-convolutions for efficient video processing," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 2695–2704.

[8] T. Verelst and T. Tuytelaars, "Dynamic convolutions: Exploiting spatial sparsity for faster inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2320–2329.

[9] F. Li, G. Li, X. He, and J. Cheng, "Dynamic dual gating neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 5310–5319.

[10] M. Ren, A. Pokrovsky, B. Yang, and R. Urtasun, "Sbnet: Sparse blocks network for fast inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 8711–8720.

[11] Z. Xie, Z. Zhang, X. Zhu, G. Huang, and S. Lin, "Spatially adaptive inference with stochastic feature sampling and interpolation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 531–548.

[12] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, "Bisenet: Bilateral segmentation network for real-time semantic segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 334–349.

[13] C. Wang, D. Xu, Y. Zhu, R. M. Martin, C. Lu, L. Fei-Fei, and S. Savarese, "Densefusion: 6d object pose estimation by iterative dense fusion," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 3343–3352.

[14] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 13 438–13 444.

[15] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "PAMPC: perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 2018, pp. 1–8. [Online]. Available: https://doi.org/10.1109/IROS.2018.8593739

[16] D. Falanga, K. Kleber, and D. Scaramuzza, "Dynamic obstacle avoidance for quadrotors with event cameras," *Sci. Robotics*, vol. 5, no. 40, p. 9712, 2020. [Online]. Available: https://doi.org/10.1126/scirobotics.aaz9712

[17] D. Hanover, A. Loquercio, L. Bauersfeld, A. Romero, R. Penicka, Y. Song, G. Cioffi, E. Kaufmann, and D. Scaramuzza, "Autonomous drone racing: A survey," *IEEE Trans. Robotics*, vol. 40, pp. 3044–3067, 2024. [Online]. Available: https://doi.org/10.1109/TRO.2024.3400838

[18] L. Cavigelli, P. Degen, and L. Benini, "Cbinfer: Change-based inference for convolutional neural networks on video data," in *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*, 2017, pp. 1–8.

[19] M. Parger, C. Tang, T. Neff, C. D. Twigg, C. Keskin, R. Wang, and M. Steinberger, "Motiondeltacnn: Sparse CNN inference of frame differences in moving camera videos with spherical buffers and padded convolutions," in *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, 2023, pp. 17 246–17 255.

[20] K. Wang, S. Yang, J. Zhao, W. Ding, Q. Chen, J. Leng, and M. Guo, "Sparsetem: Boosting the efficiency of cnn-based video encoders by exploiting temporal continuity," in *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*, 2025, pp. 246–256.

[21] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. P. Vetrov, and R. Salakhutdinov, "Spatially adaptive computation time for residual networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1790–1799.

[22] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, "2d human pose estimation: New benchmark and state of the art analysis," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 3686–3693.

[23] R. Liu, Y. Leng, S. Tian, S. Hu, C. R. Chen, and S. Yao, "Dynaspa: Exploiting spatial sparsity for efficient dynamic DNN inference on devices," in *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 2024, pp. 422–435.

[24] Spconv Contributors, "Spconv: Spatially sparse convolution library," https://github.com/traveller59/spconv, 2022.

[25] H. Tang, S. Yang, Z. Liu, K. Hong, Z. Yu, X. Li, G. Dai, Y. Wang, and S. Han, "Torchsparse++: Efficient training and inference framework for sparse convolution on gpus," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2023, pp. 225–239.

[26] H. Tang, Z. Liu, X. Li, Y. Lin, and S. Han, "TorchSparse: Efficient point cloud inference engine," in *Proceedings of Machine Learning and Systems (MLSys)*, 2022.

[27] J. Yang, C. Giannoula, J. Wu, M. Elhoushi, J. Gleeson, and G. Pekhimenko, "Minuet: Accelerating 3d sparse convolutions on gpus," in *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*, 2024, pp. 786–802.

[28] C. B. Choy, J. Gwak, and S. Savarese, "4D spatio-temporal convnets: Minkowski convolutional neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[29] B. Graham, M. Engelcke, and L. van der Maaten, "3D semantic segmentation with submanifold sparse convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[30] R. Fan, W. Wang, and X. Chu, "Dtc-spmm: Bridging the gap in accelerating general sparse matrix multiplication with tensor cores," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafrir, Eds. ACM, 2024, pp. 253–267. [Online]. Available: https://doi.org/10.1145/3620666.3651378

[31] H. Peng, X. Xie, K. Shivdikar, M. A. Hasan, J. Zhao, S. Huang, O. Khan, D. R. Kaeli, and C. Ding, "Maxk-gnn: Extremely fast GPU kernel design for accelerating graph neural networks training," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafrir, Eds. ACM, 2024, pp. 683–698. [Online]. Available: https://doi.org/10.1145/3620665.3640426

[32] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse GPU kernels for deep learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[33] A. K. Mishra, J. A. Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius, "Accelerating sparse deep neural networks," *CoRR*, vol. abs/2104.08378, 2021.

[34] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. P. Amarasinghe, "The tensor algebra compiler," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, 2017.

[35] Z. Ye, R. Lai, J. Shao, T. Chen, and L. Ceze, "SparseTIR: Composable abstractions for sparse compilation in deep learning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[36] Y. Gao, B. Zhang, X. Qi, and H. K. So, "DPACS: hardware accelerated dynamic neural network pruning through algorithm-architecture co-design," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, T. M. Aamodt, N. D. E. Jerger, and M. M. Swift, Eds. ACM, 2023, pp. 237–251. [Online]. Available: https://doi.org/10.1145/3575693.3575728

[37] F. Paredes-Vallés and G. C. H. E. de Croon, "Back to event basics: Self-supervised learning of image reconstruction for event cameras via photometric constancy," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 3445–3454.

[38] A. Z. Zhu, D. Thakur, T. Özaslan, B. Pfrommer, V. Kumar, and K. Daniilidis, "The multi vehicle stereo event camera dataset: An event camera dataset for 3d perception," *CoRR*, vol. abs/1801.10202, 2018.

[39] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[40] A. Milan, L. Leal-Taixé, I. D. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *CoRR*, vol. abs/1603.00831, 2016.

[41] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," *CoRR*, vol. abs/1410.0759, 2014.

[42] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and D. Lin, "Mmdetection: Open mmlab detection toolbox and benchmark," *CoRR*, vol. abs/1906.07155, 2019.